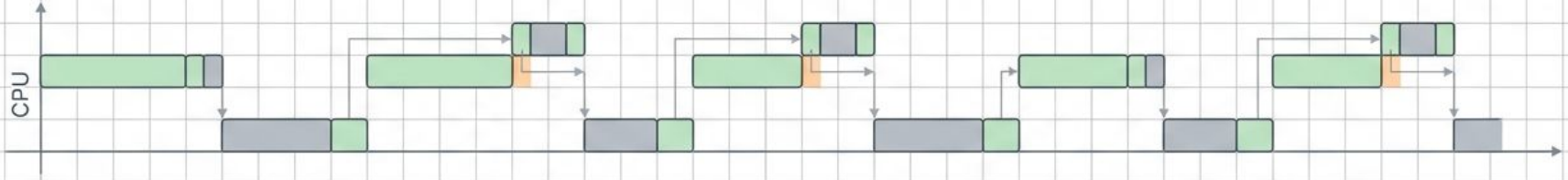
Context-Aware Deadline Scheduling Under Memory Contention

Bridging the gap between theoretical schedulability and runtime determinism in embedded systems.



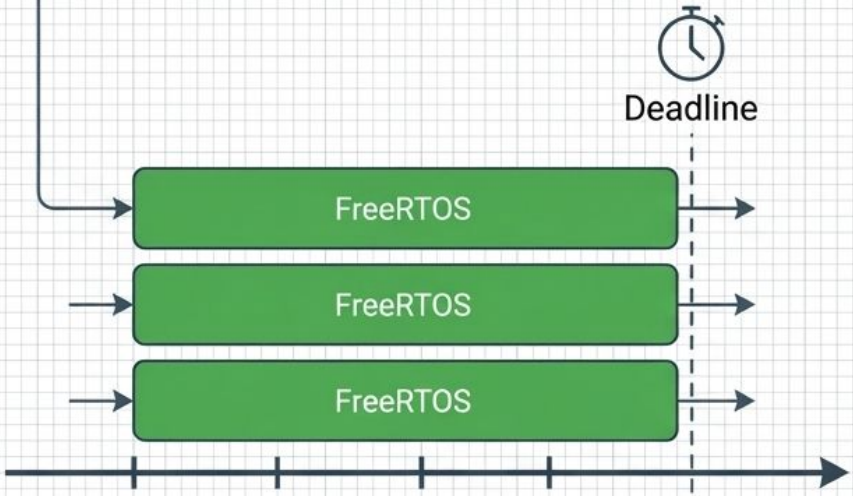
Context:	CMP720 Embedded System Design
Track:	Optimization

Team:	Umut Kalman & Eren Karadağ

NotebookLM

THE ASSUMPTION

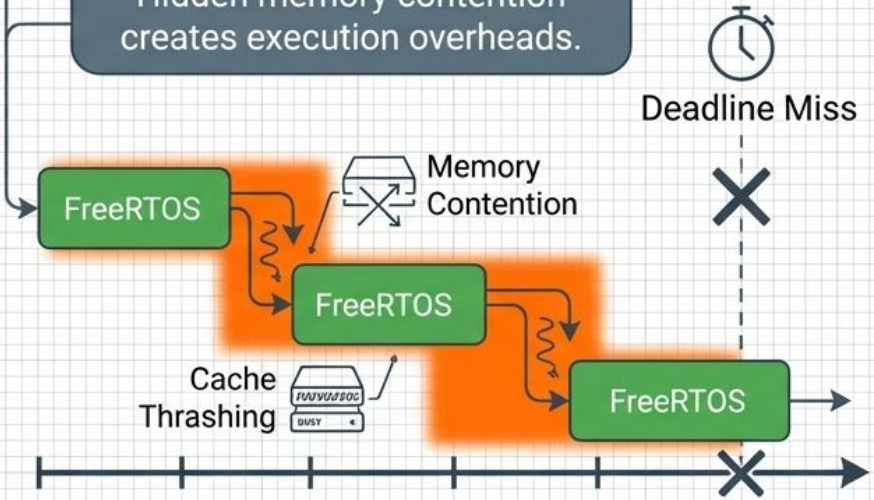
Standard schedulers perfectly optimize CPU usage and deadlines.



THE REALITY

Mixed-criticality workloads share constrained memory buses.

Hidden memory contention creates execution overheads.



THE RESULT: Systems proven schedulable on paper suffer from jitter and sporadic deadline misses in deployment.

NotebookLM

Rate Monotonic (RM)

- ✔ Static priority based on task frequency
- ✔ Simple RTOS implementation
- ✔ Lower maximum CPU utilization boundary

Earliest Deadline First (EDF)

- ✔ Dynamic priority based on closest absolute deadline
- ✔ Theoretically allows up to 100% uniprocessor utilization
- ✔ Assumes perfect hardware isolation

The Baseline Choice: EDF is theoretically optimal but its logic hides memory-system overheads—making it the perfect target for a context-aware wrapper.

NotebookLM

Related Work & Research Gap

System-Level Control

OS-level memory bandwidth isolation (e.g., MemGuard, PALLOC)

PREM / Memory-Centric

Making memory execution phases scheduler-visible

Delay Analysis (CRPD)

Modeling cache preemption delays within theoretical EDF analysis

Online / Offline Slack

Reclaiming unused execution time primarily for soft real-time optimization

The Research Gap: A lightweight, single-core RTOS wrapper combining both memory awareness and slack awareness without requiring heavy hardware or kernel changes.

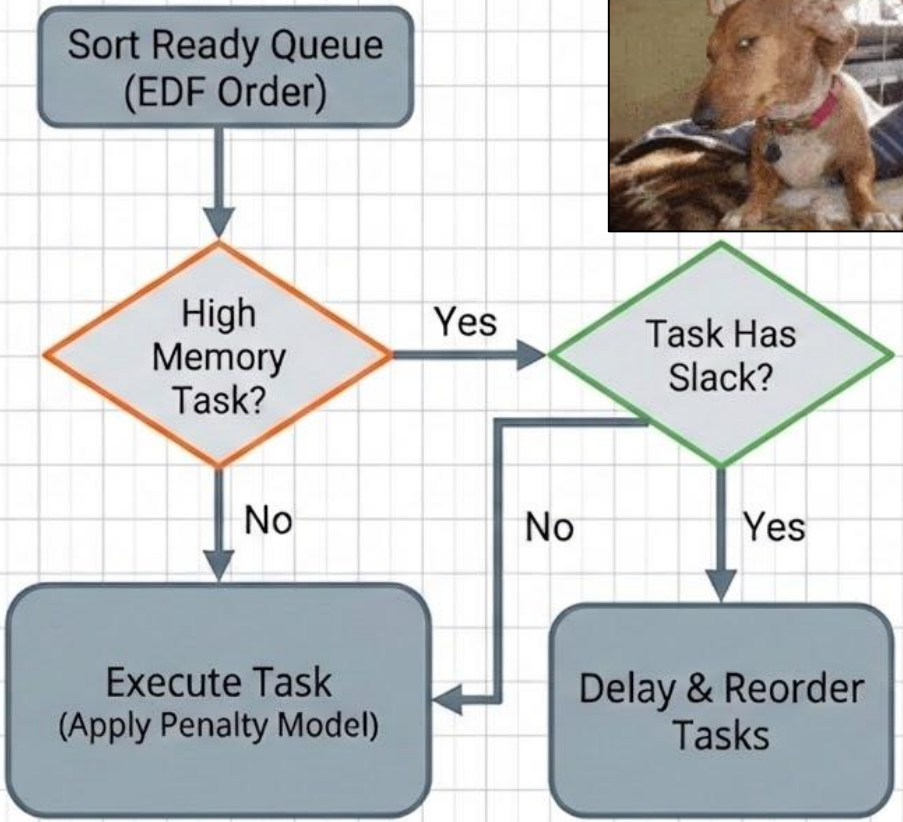
NotebookLM

Proposed Logic: Slack-Aware EDF Wrapper

Core Concept: A user-space FreeRTOS wrapper.

Base Rule: Always defaults to standard EDF to guarantee baseline safety.

The Heuristic: Evaluates task memory intensity, predicts contention via penalty model, and checks dynamic slack time.



NotebookLM

Mathematical Formulation

$$P(j_a, j_b) = \alpha M_a M_b$$

Objective: Minimize pairwise cumulative memory penalty across consecutive tasks.

$$f_b \leq d_b$$

Constraint 1 (Compliance): Completion time must never exceed the strict absolute deadline.

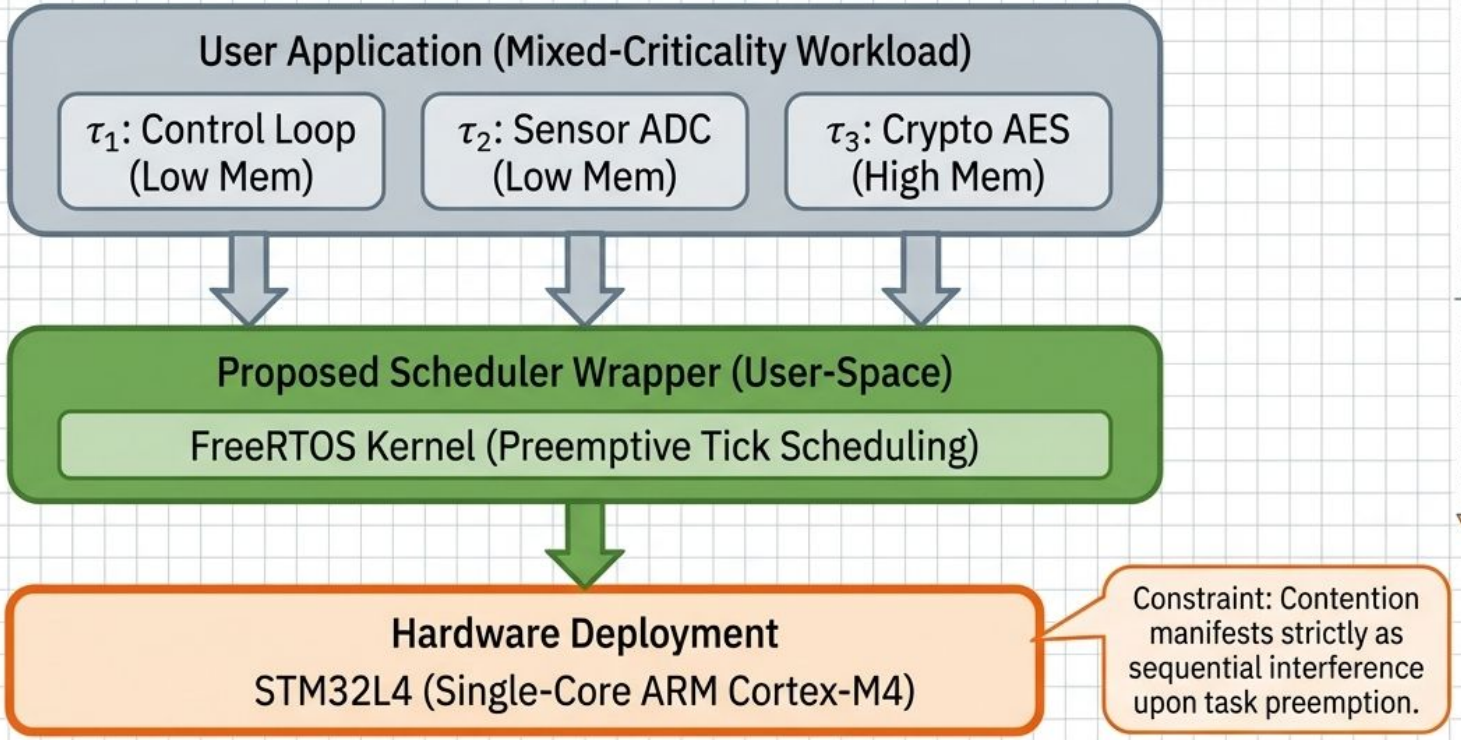
$$Slack_j(t) = d_j - t - C_{rem,j}(t)$$

Constraint 2 (**Slack Proxy**): Calculates available safe delay time before execution is critical.

The Golden Rule: High-memory tasks are only deferred if Slack > 0.

NotebookLM

System Architecture & Design



NotebookLM

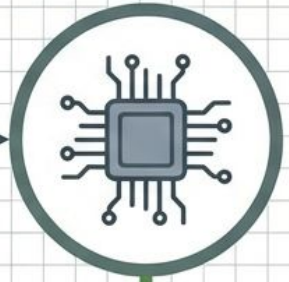
Methodology & Validation Pipeline

1. Workload Profiling

Measure isolated Worst-Case Execution Time (WCET) on physical hardware using ARM DWT cycle counters.

2. Offline ACO Generation

A Python-based Ant Colony Optimization solver determines the theoretical optimal schedule to act as the ultimate ground-truth baseline.



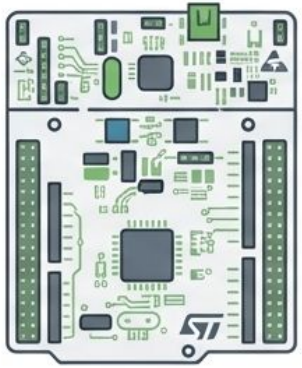
3. Online Implementation

Deploy the slack-aware wrapper on the MCU. Compare runtime performance against both the offline ACO baseline and pure EDF.

NotebookLM

Experimental Setup & Evaluation

Hardware & Environment



- **Platform:** STM32L452 Nucleo-64
- **Timing Tool:** ARM Data Watchpoint and Trigger (DWT) cycle counter
- **Workload Model:** τ_1 (Control), τ_2 (Sensor), τ_3 (Crypto)

Evaluation Metrics

Deadline Miss Ratio (DMR)

Hard real-time safety check.

Task Jitter

Variance in response time for critical control tasks.

Total Contention Penalty

Aggregate memory delay over the hyperperiod.

Scheduler Overhead

Execution cost measured in precise DWT cycles.

90°

Pure EDF vs. Proposed Wrapper vs. ACO Baseline

NotebookLM

Expected Contributions & Conclusion

The Problem Solved:
Standard uniprocessor schedulers ignore microarchitectural memory interference.

The Contribution: A lightweight, embedded-friendly EDF wrapper that minimizes consecutive memory penalties while strictly preserving deadline safety.

Reliability

Guaranteed safe fallback to pure EDF when scheduling slack is zero.

Performance

Tangible reduction in timing jitter for critical single-core control loops.

Efficiency

Low O(n) ready-queue check overhead suitable for highly constrained MCU targets.

90°

NotebookLM